

VAPT SANITIZER

v1.0.0

Technical User Guide

Local-first security gateway for authorized VAPT -> AI workflows

LOCAL-FIRST | ENCRYPTED MAPPING | PASS / REVIEW / BLOCKED

Stable release - validated baseline

181/181 tests OK | Auto-detection 10/10 PASS | Burp BUILD SUCCESSFUL

SHA-256 stable ZIP

2ee9928dddb789f1f5fda3479ad86e0c0540deabbe3bf97c5ed82cb666479c84

Indice

1. Panoramica e obiettivi
2. Modello di sicurezza
3. Architettura
4. Profili supportati e auto-detection
5. Installazione e validazione
6. Uso della CLI
7. AI Handoff e Security Gate
8. Engagement Mapping Vault
9. Integrazione Burp Suite
10. Workflow operativi consigliati
11. Policy e modalità di protezione
12. Troubleshooting
13. Sicurezza operativa e retention
14. Stato della release e validazione
- A. Appendice - Comandi rapidi
- B. Appendice - Checklist per un nuovo assessment

Destinatari: Pentester, VAPT engineer, Red Team operator e colleghi tecnici che devono utilizzare sistemi AI senza trasferire direttamente identificativi reali e secret del cliente.

Ambito: La guida descrive la Stable v1.0.0 validata. Non sostituisce NDA, regole di ingaggio, data handling policy o requisiti contrattuali del cliente.

1. Panoramica e obiettivi

VAPT Sanitizer è un gateway locale pensato per inserire un confine di sicurezza tra gli output tecnici di un assessment autorizzato e un sistema di Intelligenza Artificiale utilizzato come assistente analitico.

L'obiettivo non è nascondere indiscriminatamente tutto il contenuto: il valore del tool sta nel preservare porte, servizi, versioni, path, evidenze, CVE e relazioni tecniche, proteggendo allo stesso tempo identificativi e secret che non devono essere trasferiti all'esterno.

1.1 Il problema operativo

```
Nmap scan report for srv-app.cliente.local (10.20.30.45)
Authorization: Bearer eyJ...
user=m.rossi email=mario.rossi@cliente.example
AWS_SECRET_ACCESS_KEY=...
```

Un LLM può aiutare a interpretare questi dati, ma l'input originale può contenere elementi che non devono essere condivisi. VAPT Sanitizer trasforma il dataset mantenendo il contesto necessario all'analisi.

```
srv-app.cliente.local -> [HOSTNAME_001]
10.20.30.45 -> [PRIVATE_IP_001]
mario.rossi@... -> [EMAIL_001]
password=... -> password=[PASSWORD_REDACTED]
```

1.2 Cosa offre la v1.0

- Sanitizzazione local-first di file, stdin e clipboard.
- Auto-detection del profilo o selezione esplicita.
- Pseudonimizzazione stabile dei valori correlabili.
- Redazione irreversibile dei secret obbligatori.
- Security Gate PASS / REVIEW / BLOCKED.
- Prompt AI-ready provider-neutral senza invio automatico.
- Engagement Mapping Vault cifrato e persistente.
- Correlazione CLI <-> Burp e cross-tool.
- Estensione Burp Suite con preview, policy, engagement e AI Handoff.
- Fixture sintetici, test di regressione e release check.

Principio operativo: L'AI è l'assistente analitico; VAPT Sanitizer è il security boundary; il penetration tester rimane il decisore finale.

2. Modello di sicurezza

2.1 Pseudonimizzazione

I valori che devono rimanere correlabili vengono sostituiti con placeholder leggibili. Se l'Engagement Mapping Vault è attivo, lo stesso valore reale mantiene lo stesso placeholder tra file, processi e strumenti diversi nello stesso engagement.

```
10.20.30.45 -> [PRIVATE_IP_001]
srv-app.interno.local -> [HOSTNAME_001]
user@cliente.example -> [EMAIL_001]
```

2.2 Redazione

I secret non devono essere recuperabili. Vengono quindi redatti e non sono inseriti nel Mapping Vault.

```
password=SuperSecret -> password=[PASSWORD_REDACTED]
Authorization: Bearer eyJ... -> Authorization: [BEARER_TOKEN_REDACTED]
PRIVATE KEY block -> [PRIVATE_KEY_REDACTED]
```

2.3 Mandatory secret protections

- PASSWORD
- BEARER_TOKEN
- JWT
- PRIVATE_KEY
- AWS_ACCESS_KEY
- AWS_SECRET_KEY
- AWS_SESSION_TOKEN
- API_KEY
- SESSION
- AUTH_CREDENTIAL
- DATABASE_PASSWORD
- PROVIDER_TOKEN
- GENERIC_SECRET

Invariante: Una policy YAML non può trasformare un mandatory secret in un dato liberamente esportabile. La redazione dei secret resta una barriera non indebolibile dal file di policy.

2.4 Trust boundary

Il vault cifra i mapping a riposo e separa la chiave dal ciphertext. Questo riduce esposizioni accidentali e protegge backup/repository, ma non difende da un attaccante che abbia già compromesso l'account locale e possa leggere sia master.key sia mapping.enc.

3. Architettura

INPUT Nmap / Burp / Gobuster / Nuclei / Windows / Linux / AWS / file / clipboard
CONTEXT Profile detection e tool metadata
DETECTION Detectors + policy engine
TRANSFORM Pseudonimizzazione / redazione
VAULT Mapping locale cifrato per engagement - solo valori pseudonimizzati
GATE PASS / REVIEW / BLOCKED
HANDOFF Prompt AI-ready provider-neutral
CLIPBOARD Copia locale verificata
OPERATOR Paste manuale nel browser AI e risoluzione locale dei placeholder

Nessun invio automatico: Nel workflow AI Handoff standard la generazione del prompt non esegue richieste di rete. Il trasferimento al servizio AI resta un'azione manuale dell'operatore.

4. Profili supportati e auto-detection

Profilo	Input tipico	Comportamento principale
GENERIC	Log e contenuti misti	Hostname, IP, username, email, secret; preserva contesto tecnico generale
NMAP	Output Nmap	Target host/IP/MAC; preserva porte, servizi, versioni, CVE, NSE
BURP	HTTP request/response	Host, identity, auth header, session, secret; preserva struttura HTTP
GOBUSTER	Directory/file enumeration	Target metadata; preserva path, file, status code e wordlist
NUCLEI	Nuclei findings	Target identifiers; preserva template, severity, CVE e riferimenti pubblici
WINDOWS	Windows/AD enumeration	Hostname, SID, domain, IP; preserva contesto Windows standard
LINUX	Linux enumeration/log	Hostname, user ambientali, IDs, IP/IPv6/MAC; preserva utenti di servizio standard
AWS	Cloud/AWS output	Account/resource/IAM identifiers; redige credenziali e token AWS
SECRETS	Secret-heavy data	Password, token, API key, private key, DSN, credential material
PII	Dati personali	Nome, telefono, DOB, tax/fiscal ID, address, city, postal, IBAN, username

4.1 Auto-detection

Se non si specifica `--profile`, il programma usa il profilo auto. La release contiene fixture sintetici dedicati e la matrice di validazione ufficiale è 10/10 PASS.

```
./venv/bin/vapt-sanitize scan.txt -o scan.sanitized
# profile selected automatically
```

4.2 Quando forzare il profilo

Usare un profilo esplicito quando l'input è ambiguo, contiene più formati non standard o si vuole un comportamento deterministico per una procedura documentata.

```
./venv/bin/vapt-sanitize scan.txt --profile nmap
```

5. Installazione e validazione

5.1 Requisiti

Voce	Dettaglio
Python	3.11 o superiore con supporto venv

Voce	Dettaglio
Dipendenze Python	PyYAML 6.0.3; cryptography 46.0.4
Clipboard Linux	xclip / xsel / wl-copy opzionale
Burp build	Java 17+, unzip, curl oppure wget
Gradle	Non richiesto a livello di sistema; bootstrap Gradle 8.14.3 incluso

5.2 Installazione Stable

Nota sul nome directory: Lo ZIP Stable è identico byte-per-byte alla RC2 validata, quindi estrae ancora vapt-sanitize-v1.0.0-rc2/. L'applicazione riporta comunque versione 1.0.0.

```
unzip vapt-sanitize-v1.0.0.zip
cd vapt-sanitize-v1.0.0-rc2
./scripts/install.sh
```

Lo script crea .venv, installa il progetto in editable mode con le dipendenze congelate, verifica il backend clipboard e lancia la suite di regressione.

5.3 Verifica

```
./venv/bin/python -m vapt_sanitize --version
./scripts/release-check.sh
```

```
VAPT Sanitizer 1.0.0
181 tests OK
Auto-detection result: 10/10 PASS
[+] Release checks passed.
```

5.4 Directory operativa per Burp

La UI Burp usa come default ~/vapt-sanitize e ~/vapt-sanitize/.venv/bin/python. Per una installazione versionata si può usare un symlink/rename operativo oppure avviare Burp con variabili d'ambiente esplicite.

```
export VAPT_SANITIZE_HOME="$HOME/vapt-sanitize-v1.0.0-rc2"
export VAPT_SANITIZE_PYTHON="$VAPT_SANITIZE_HOME/.venv/bin/python"
burpsuite
```

6. Uso della CLI

6.1 Comandi base

```
# File -> output sanitizzato
./venv/bin/vapt-sanitize scan.txt -o scan.sanitized

# Stdin
cat scan.txt | ./venv/bin/vapt-sanitize -

# Profilo esplicito
./venv/bin/vapt-sanitize scan.txt --profile nmap

# Decisioni spiegate
./venv/bin/vapt-sanitize scan.txt --explain
```

6.2 Clipboard

```
# Legge la clipboard e la sostituisce con il contenuto sanitizzato
./venv/bin/vapt-sanitize --clipboard

# Legge la clipboard e la sostituisce con il prompt AI-ready
./venv/bin/vapt-sanitize --clipboard --ai-prompt
```

6.3 Prompt AI da file o stdin

```
./venv/bin/vapt-sanitize scan.txt \
  --ai-prompt \
  --ai-clipboard
```

6.4 Task AI disponibili

- analyze-security
- explain-response
- find-attack-surface
- suggest-next-tests
- custom

```
./venv/bin/vapt-sanitize burp.txt \
  --ai-prompt --ai-clipboard \
  --task custom \
  --question "Quali test di autorizzazione conviene prioritizzare?"
```

Source label: Il campo Source del prompt viene derivato dal profilo/tool; può essere sovrascritto con --source senza includere automaticamente l'ID engagement.

7. AI Handoff e Security Gate

Il Security Gate viene valutato prima della generazione dell'AI-ready prompt. La sua funzione è impedire che l'handoff diventi un semplice "copy all" automatico.

Stato	Significato	Comportamento
PASS	Nessuna regola implementata richiede intervento	Handoff consentito
REVIEW	Serve controllo e approvazione manuale	CLI: --review-approved; Burp: dialog di conferma
BLOCKED	Il boundary non è soddisfatto	Handoff rifiutato; non bypassabile con review approval

```
./venv/bin/vapt-sanitize input.txt \
  --ai-prompt --ai-clipboard \
  --review-approved
```

Interpretazione corretta: PASS non è una garanzia matematica di anonimizzazione universale: significa che i controlli implementati non hanno rilevato condizioni che richiedono REVIEW o BLOCKED. La preview manuale resta obbligatoria a livello operativo.

7.1 Forma del prompt

```
You are assisting with an authorized security assessment.
Source: Nmap enumeration

Analyze this sanitized security-testing data...
Do not attempt to reconstruct, infer, or guess original values.

--- SANITIZED DATA ---
Nmap scan report for [HOSTNAME_001] ([PRIVATE_IP_001])
```

8. Engagement Mapping Vault

Il Mapping Vault risolve il problema principale dei workflow multi-tool: senza uno stato persistente, due processi distinti potrebbero assegnare lo stesso placeholder a valori reali diversi o placeholder diversi allo stesso host. Con un engagement, il mapping è persistente e isolato per assessment.

8.1 Creazione

```
./.env/bin/vapt-sanitize engagement init ACME-2026-EXTERNAL
```

L'ID deve essere lungo 1-64 caratteri, iniziare con una lettera o numero e può contenere lettere, numeri, punto, underscore e trattino.

8.2 Uso cross-tool

```
./.env/bin/vapt-sanitize scan.txt \
--engagement ACME-2026-EXTERNAL \
--ai-prompt --ai-clipboard
```

```
Nmap: 10.20.30.45 -> [PRIVATE_IP_001]
Burp: 10.20.30.45 -> [PRIVATE_IP_001]
Nuclei: 10.20.30.45 -> [PRIVATE_IP_001]
```

8.3 Show e resolve

```
./.env/bin/vapt-sanitize engagement show ACME-2026-EXTERNAL

./.env/bin/vapt-sanitize engagement resolve \
ACME-2026-EXTERNAL \
"[PRIVATE_IP_001]" "[HOSTNAME_001]"
```

8.4 Percorsi e protezioni

```
~/.config/vapt-sanitize/master.key
~/.local/share/vapt-sanitize/engagements/<ID>/mapping.enc
~/.local/share/vapt-sanitize/engagements/<ID>/metadata.json
~/.local/share/vapt-sanitize/engagements/<ID>/mapping.lock
```

- Chiave e ciphertext sono separati.
- Il mapping è cifrato a riposo con authenticated encryption tramite cryptography/Fernet.
- Le directory/file vengono creati con permessi restrittivi quando il sistema operativo lo consente.
- Il lock per engagement serializza gli update concorrenti.
- I secret redatti non entrano nel vault.
- La perdita di master.key rende non decifrabili i mapping esistenti.

Backup: Se la correlazione deve sopravvivere a reinstallazione o sostituzione della workstation, conservare un backup offline protetto di master.key insieme a una strategia di retention approvata.

9. Integrazione Burp Suite

9.1 Build

```
./scripts/build-burp.sh
```

Il bootstrap incluso usa Gradle 8.14.3 verificato SHA-256. Non è necessario installare Gradle dai repository Kali.

```
burp-extension/build/libs/vapt-sanitize-burp-1.0.0.jar
```

9.2 Installazione in Burp

1. Aprire Burp Suite.
2. Extensions -> Installed -> Add.
3. Scegliere extension type Java.
4. Selezionare vapt-sanitize-burp-1.0.0.jar.
5. Aprire la tab VAPT Sanitizer e verificare lo stato runtime.

9.3 Engagement panel

```
Engagement Mapping Vault
Active engagement: [ (None - stateless) v ] [ Refresh ]
```

Ogni nuova sessione parte volutamente da Stateless. Questo riduce il rischio di contaminare un progetto nuovo con il cliente precedentemente selezionato. Click su Refresh per rileggere i vault locali e selezionare l'engagement corretto.

9.4 Context menu e preview

```
VAPT Sanitizer
Engagement: ACME-2026-EXTERNAL
Sanitize request -> preview
Sanitize response -> preview
Sanitize selection -> preview
Policy
```

La preview mostra Security Gate, input type, policy, findings, engagement, detections e contenuto sanitizzato. I pulsanti principali sono Copy sanitized e Copy AI Prompt. In stato REVIEW viene richiesta conferma; in BLOCKED l'azione viene disabilitata.

9.5 Correlazione CLI <-> Burp

```
CLI/Nmap: srv-app.interno.local -> [HOSTNAME_001]
Burp: Host: srv-app.interno.local -> Host: [HOSTNAME_001]
```

9.6 Runtime path

```
export VAPT_SANITIZE_HOME="$HOME/vapt-sanitize-v1.0.0-rc2"
export VAPT_SANITIZE_PYTHON="$VAPT_SANITIZE_HOME/.venv/bin/python"
burpsuite
```

10. Workflow operativi consigliati

10.1 Assessment standard

1. Creare un engagement coerente con cliente e scope.
2. Eseguire normalmente gli strumenti VAPT autorizzati.
3. Salvare/catturare gli output.
4. Sanitizzare sempre usando lo stesso engagement.
5. Controllare profile detection, findings e Security Gate.
6. Generare il prompt AI-ready.
7. Effettuare una review visiva del prompt.
8. Incollare manualmente il prompt nel servizio AI scelto.
9. Leggere l'analisi usando i placeholder come identità persistenti.
10. Usare engagement resolve per tornare ai valori reali quando serve.
11. Continuare l'assessment sul dato reale e documentare le evidenze.

10.2 Convenzione engagement

CLIENTE - ANNO - SCOPE

ACME - 2026 - EXTERNAL
ACME - 2026 - WEBAPP
ACME - 2026 - INTERNAL

Regola fondamentale: Non riutilizzare lo stesso engagement per clienti differenti. Non usare engagement diversi per file dello stesso assessment se si vuole mantenere la correlazione dei placeholder.

10.3 Quando usare Stateless

- Fixture e test sintetici isolati.
- Output singolo senza necessità di correlazione.
- Verifiche rapide in cui il mapping persistente non serve.

11. Policy e modalità di protezione

Voce	Dettaglio
default.yml	Bilanciata: pseudonimizza identificatori correlabili e redige secret/credit card.
aggressive_pii.yml	Redige più dati personali, inclusi diversi campi PII e username in più profili.
strict_llm.yml	Aumenta il blocco per handoff LLM: preserved finding e no-findings diventano BLOCKED; redige email/username in profili rilevanti.

```
./venv/bin/vapt-sanitize input.txt \
--policy policies/strict_llm.yml \
--ai-prompt
```

Le policy modificano il trattamento di categorie non obbligatorie e la logica del gate, ma non possono disabilitare la redazione dei mandatory secret.

12. Troubleshooting

Engagement vaults require the cryptography package

Il processo che usa il vault non sta eseguendo l'ambiente virtuale corretto. Verificare `./venv/bin/python` e reinstallare con `./scripts/install.sh` o `./venv/bin/python -m pip install -e .`

Burp non mostra l'engagement

Crearlo dalla CLI, premere Refresh, verificare che Burp e CLI usino le stesse directory config/data XDG o `VAPT_SANITIZE_*`.

CLI e Burp assegnano placeholder diversi

Controllare engagement ID, master.key e data root. I due processi devono condividere lo stesso vault.

Clipboard mode fallisce

Installare xclip/xsel/wl-copy e assicurarsi che la sessione grafica esponga il clipboard corretto.

Security Gate = REVIEW

Ispezionare il contenuto; se approvato usare `--review-approved` oppure confermare il dialog Burp.

Security Gate = BLOCKED

Non bypassare. Correggere input/policy/condizione e ripetere la sanitizzazione.

Build Burp: Gradle non trovato

Usare la Stable e `./scripts/build-burp.sh`. Il bootstrap Gradle 8.14.3 è incluso; servono Java 17+, unzip e curl/wget.

Master key not found / invalid

Non rigenerare alla cieca se esistono vault da preservare. Ripristinare il backup corretto di master.key; una chiave diversa non può decifrare mapping.enc.

```
./venv/bin/python -c "from cryptography.fernet import Fernet; print('cryptography OK')"
```

13. Sicurezza operativa e retention

13.1 File da proteggere

```
~/config/vapt-sanitize/master.key
~/local/share/vapt-sanitize/engagements/
client outputs / sanitized reports / notes correlabili
```

- Non committare key o mapping su Git.
- Non allegare il vault a ticket o chat esterne.
- Non inserire mapping o real value nel prompt AI.
- Applicare la retention prevista da contratto/NDA/policy interna.
- Considerare il mapping materiale cliente anche se cifrato.
- Proteggere la workstation: il sistema operativo è parte del trust boundary.

13.2 Limiti

- Il tool non è un DLP universale.
- Nuovi formati possono richiedere detector/profilati futuri.
- Sono possibili falsi positivi o falsi negativi.
- La preview umana resta parte del processo.
- VAPT Sanitizer non sostituisce vulnerability scanner, exploit framework, SIEM o password manager.

14. Stato della release e validazione

Voce	Dettaglio
Versione	VAPT Sanitizer 1.0.0 Stable
Regression suite	181/181 tests OK
Auto-detection fixtures	10/10 PASS
Burp build	BUILD SUCCESSFUL
JAR	vapt-sanitize-burp-1.0.0.jar
Python	>= 3.11
cryptography	46.0.4
PyYAML	6.0.3
Java	>= 17
Gradle bootstrap	8.14.3
Montoya API	2026.7

14.1 Stable artifact

vapt-sanitize-v1.0.0.zip
SHA-256:
2ee9928dddb789f1f5fda3479ad86e0c0540deabbe3bf97c5ed82cb666479c84

Golden baseline: La Stable è stata promossa byte-per-byte dalla RC2 che ha superato 181 test, 10/10 auto-detection e BUILD SUCCESSFUL. Per nuove funzionalità è preferibile partire da una versione successiva senza alterare l'artefatto v1.0.0.

Appendice A - Comandi rapidi

```
# Versione
./venv/bin/vapt-sanitize --version

# Sanitizza file
./venv/bin/vapt-sanitize input.txt -o output.sanitized

# AI prompt -> clipboard
./venv/bin/vapt-sanitize input.txt --ai-prompt --ai-clipboard

# Crea engagement
./venv/bin/vapt-sanitize engagement init CLIENTE-2026-001

# Usa engagement
./venv/bin/vapt-sanitize input.txt --engagement CLIENTE-2026-001 --ai-prompt --ai-clipboard

# Mostra mapping
./venv/bin/vapt-sanitize engagement show CLIENTE-2026-001

# Risolvi placeholder
./venv/bin/vapt-sanitize engagement resolve CLIENTE-2026-001 "[HOSTNAME_001]"

# Release check
./scripts/release-check.sh

# Build Burp
./scripts/build-burp.sh
```

Appendice B - Checklist per un nuovo assessment

- ☐ Engagement ID creato e coerente con cliente/scope.
- ☐ master.key protetta e, se richiesto, backup offline disponibile.
- ☐ Regole di ingaggio autorizzano i test previsti.
- ☐ Policy scelta consapevolmente.
- ☐ Burp avviato con VAPT_SANITIZE_HOME/PYTHON corretti se installazione versionata.
- ☐ Burp Engagement Mapping Vault impostato sul cliente corretto, non Stateless.
- ☐ Ogni AI Handoff sottoposto a review visiva.
- ☐ REVIEW approvato solo dopo controllo; BLOCKED non bypassato.
- ☐ Mapping mai incollato in AI.
- ☐ Retention del vault definita per fine assessment.

Principio finale: Preservare il contesto tecnico utile all'analisi AI senza trasformare il servizio AI nel repository dei dati reali del cliente. La correlazione resta locale; il giudizio resta umano.